An Intelligent Public Transport Delay Prediction and Route Optimization System Using Machine Learning

Jerin Philip Mathew*, Pregin Athish E*
Department of Computer Science and Engineering, School of Engineering and Technology
CHRIST (Deemed to be University), Kumbalgodu, Bengaluru – 560 074, India
Register Nos.: 2360385, 2360439

***Abstract*—Urban bus transit in fast-growing metropolitan cities suffers from poor schedule reliability caused by traffic congestion, adverse weather, monsoon delays, crowding at transfers and incidents. Most agencies still rely on GTFS (General Transit Feed Specification) timetables that ignore dynamic traffic, so commuters miss connections and wait longer at stops, and no existing solution lets agencies proactively schedule buses or manage incidents such as bus bunching. This paper presents an enterprise-level Intelligent Transportation System that fuses 13 municipal data feeds into a Kimball Star Schema data warehouse holding 729,648 trip-stop arrival records (2022–2026) from four arterial corridors in Bengaluru. Two models predict delay: a tabular XGBoost regressor (MAE 1.30 min, $R^2 = 0.986$) and a Bidirectional LSTM with Bahdanau attention (MAE 0.35 min, $R^2 = 0.997$). Cost-sensitive XGBoost and Random Forest classifiers label disruption severity with a macro F1 of 0.751 and 87.9% accuracy. The predicted delays drive a Dijkstra route-optimization engine on OpenStreetMap graphs, which on the Hebbal–Silk Board corridor cuts travel time by 82.3 minutes (75.5%) through a faster alternate route. SHAP-based explainable AI turns predictions into human-readable detour cards, and a dual-perspective dashboard serves both commuters and transit operators.**

***Index Terms*—Intelligent transportation systems, delay prediction, XGBoost, bidirectional LSTM, attention mechanism, explainable AI, SHAP, Dijkstra routing, GTFS, data warehousing.**

I. Introduction

A. Background and Motivation

In growing metropolitan cities, public bus transportation plays a vital role in the daily commute of passengers. Traffic jams, unexpected weather changes, heavy rain, accidents, crowded buses and other operational issues all affect bus travel. These problems cause bus bunching and longer commuter wait times, and delays at one stop can spread to downstream stops. Traditional public transportation scheduling is based on General Transit Feed Specification (GTFS) data [1], which cannot effectively handle dynamic traffic and environmental change.

This work explores an Intelligent Transportation System (ITS) that leverages multimodal data integration, machine learning and deep learning, geospatial routing and explainable artificial intelligence to mitigate these constraints. The system integrates 13 heterogeneous data sources—including GTFS schedules, Automatic Vehicle Location (AVL) GPS data, weather, traffic and passenger occupancy information—into a Kimball Star Schema data warehouse [7] consisting of 729,648 operational arrival records from 2022 to 2026.

For delay prediction, the project uses a hybrid approach that combines XGBoost [2] and a Bidirectional LSTM [9] with Bahdanau attention [3]. Dijkstra-based dynamic routing [5] is used to find alternative routes under different traffic situations, and SHAP-based explainable AI [4] provides comprehensible insights into predicted delays and suggested alternatives.

B. Objectives

The project has three key objectives:

1) To implement a machine learning model that analyses past transit, traffic and weather data to predict public bus delays accurately, by (i) combining traffic, weather and multimodal schedule data into a single data warehouse, (ii) modelling the sequential propagation of delays between successive bus stops, and (iii) classifying transit anomalies by the extent of their impact.
2) To optimize travel routes by recommending effective diversions based on predicted delays, by (i) developing a dynamic route optimization system that maps road networks using OpenStreetMap, (ii) comparing algorithms for multi-criteria shortest-path routing, and (iii) implementing SHAP-based explainable AI to determine and explain the primary factors that influence route recommendations.
3) To develop an interactive dashboard that shows forecasts and offers useful data, by (i) building a transit interface for both users and managers for better visualization and understanding, and (ii) evaluating regression, routing and execution performance to analyse the proposed system.

C. Delimitation of Research

The research is confined to the design, development and analysis of an intelligent transportation system prototype for dynamic route optimization and public bus delay prediction. The analysis uses an integrated five-year operational dataset spanning 2022 to 2026 and focuses on specific transit lines in Bengaluru. Data warehousing, preprocessing, feature engineering, machine- and deep-learning-based prediction, disruption

classification, dynamic routing and explainable AI are all included.

The project does not cover complete city-wide production deployment, continuous live-streaming infrastructure, large-scale fleet dispatch automation or long-term operational monitoring. Although cloud engineering and financial feasibility are evaluated, the implementation remains primarily a research prototype. Graph Neural Network-based spatial modelling, live data-stream ingestion and reinforcement-learning-based fleet dispatch optimization are identified as areas for further research.

D. Benefits of Research

Both commuters and public transport management may benefit from the proposed approach. It increases the accuracy of bus delay estimates by accounting for many operational, environmental and temporal factors, and its sequential model captures the transmission of delays from one bus stop to the next. The system also provides dynamic route optimization to deal with extreme traffic and interruptions. By highlighting significant variables such as road congestion, rainfall and peak-hour conditions, SHAP-based explanations make the model's forecasts understandable to travellers.

Commuters gain access to expected arrival times, delay severity and alternative routes, while transport operators can identify recurrent congestion locations and receive recommendations for schedule buffer adjustments. The study thus offers a comprehensive framework for predictive analysis, route optimization and explainable decision-making in intelligent public transportation.

II. Literature Review

A. Overview of Existing Transit Prediction Approaches

Predicting public transport delays has occupied transportation-engineering and computer-science researchers for more than two decades. Early transit management systems used statistical techniques such as linear regression and historical-average models. These static models are simple to compute but cannot represent the intricate interactions brought on by unexpected traffic jams or shifting weather.

With the development of machine learning, researchers adopted tree-based and related algorithms including Random Forest, Support Vector Regression and Gradient Boosted Decision Trees. By processing inputs such as time of day, day of week and weather conditions, these tabular models greatly increased the accuracy of delay prediction. However, conventional tabular models treat each bus-stop arrival as an independent event, ignoring the fact that a bus moves sequentially along a route and carries its prior delay forward.

More recently, deep learning techniques—Recurrent Neural Networks, Long Short-Term Memory (LSTM) networks [9] and Graph Neural Networks—have been applied to delay prediction. LSTMs are particularly good at identifying time-series trends across bus stops. Nevertheless, deep learning models are computationally costly and frequently function as "black boxes", offering no justification for a particular delay forecast.

B. Global Public Transport Delay Datasets

To place the dataset used in this project in context, a comparative survey of available public transport delay-prediction datasets was performed. Table I contrasts the BMTC dataset created for this project with benchmark datasets used in international transportation research.

C. Old versus New Dataset Comparison

During the project, a pilot dataset was upgraded into an integrated data warehouse. The comparison below demonstrates the need for the new integrated dataset in machine learning research.

- **Record temporal span.** The previous dataset had approximately 50,000 records for a single three-month period in 2023. The new integrated dataset includes 729,648 arrival records covering five whole years, from 2022 to 2026.
- **Feature depth.** The original dataset contained 12 columns (stop name, scheduled time, actual time and delay). The new dataset contains 35 features, including OpenMeteo weather data (temperature, humidity, precipitation, weather codes), TomTom traffic congestion indices, occupancy percentages from an Automated Passenger Counter (APC), official Karnataka holiday flags, and lag times for upstream stops.
- **Database architecture.** The earlier dataset used flat CSV files that often duplicated data or contained missing entries. The new dataset is organized as an enterprise Kimball Star Schema data warehouse in SQLite (`bmtc_data_warehouse.db`) with one central fact table (`fact_integrated_bus_operations`), four dimension tables (`dim_bus`, `dim_stop`, `dim_calendar`, `dim_route`) and analytic views (`vw_route_performance`, `vw_stop_occupancy`).

D. Comparative Literature Matrix

Table II compares key representative published studies against the proposed system.

E. Summary of Core Research Gaps

Reviewing public transport prediction systems reveals several research gaps that limit commuter trust and operational efficiency. Existing models function as "black boxes", offering incomplete, non-actionable delay forecasts without explaining the underlying environmental or traffic-related causes. These systems often rely on static timetables and isolated stop models, which do not account for how live telemetry noise affects prediction accuracy or how delays propagate sequentially downstream. Current platforms also suffer considerable processing latency and cannot offer dynamic graph-rerouting engines during severe disruptions such as heavy weather or localized gridlock. They merely report that a transit vehicle is delayed, rather than giving commuters or depot managers real-time, mathematically optimized diversion choices.

TABLE I
GLOBAL TRANSIT DATASETS COMPARATIVE ANALYSIS

Dataset	City / Location	Records	Primary Features	Limitations
CTA Bus Tracker	Chicago, USA	∼250,000 trips	GPS, schedule, timestamp	Lacks weather and passenger count
MTA Bus Time	New York, USA	∼1,200,000 pings	AVL GPS coordinates	High GPS noise, no schedule context
Dublin Bus GTFS-R	Dublin, Ireland	∼500,000 records	Delay seconds, stop ID	Short duration (1 month only)
TfL Open Data	London, UK	∼2,000,000 events	Tube/bus arrival predictions	Complex APIs, static features
BMTC Star Schema (this project)	Bengaluru, India	**729,648 trips**	13 fused feeds (GTFS, weather, traffic, APC)	5-year multi-year integrated star-schema warehouse

TABLE II
COMPARATIVE LITERATURE REVIEW MATRIX

Study	Methodology	Dataset	Strengths	Limitations
Zhang et al. (2020)	Random Forest regressor	City GPS logs	Simple, fast training	Ignores sequential propagation and weather
Liu & Chen (2021)	Standard LSTM	GTFS schedule + telemetry	Captures time-series memory	Black-box; static routing
Wang et al. (2022)	Graph Convolutional Network (GCN)	Taxi and bus trajectories	Models spatial road-graph relationships	High computational complexity; no XAI
Kumar & Devi (2023)	XGBoost regressor	Municipal transit CSV	Handles non-linear tabular features well	No dynamic routing or telemetry imputation
Proposed system (this work)	BiLSTM + attention, XGBoost, Dijkstra routing	729,648 records fused from 13 feeds (2022–2026)	MAE 0.35 min; SHAP XAI cards; 75.5% travel-time reduction	Requires structured multi-source data-ingestion pipeline

III. METHODOLOGY

A. Problem Formulation

Urban public transport is disrupted by traffic, weather, accidents and crowds. Current management systems use schedules that cannot respond to real-time incidents or suggest alternate routes. Existing machine learning methods are complex and vulnerable when GPS locations are lost; they fail to treat bus stops as individual entities whose delays propagate to downstream stops; and none of them gives actionable recommendations to commuters or transport administrators.

To build a mathematical framework for delay prediction, transit disruptions are deconstructed into four real-world components:

- **Traffic congestion delay** (D_{traffic}): delay caused by vehicular volume exceeding road capacity, measured using real-time traffic speed indices and congestion metrics.
- **Meteorological delay** (D_{weather}): delay from rain, poor visibility or extreme weather that lowers vehicle speeds and increases stopping distances.
- **Operational and dwelling delay** (D_{dwelling}): delay at bus stops due to high boarding and alighting volumes, crowding or ticketing.
- **Cascading downstream delay** (D_{cascade}): delay accumulated at upstream stop $k-1$ that carries over to stop k because of the vehicle's continuous movement along its route.

The total delay at stop-sequence position k is a function of these four components and is modelled additively:

$$D_{\text{total}}(k) = f\big(D_{\text{traffic}}(k), D_{\text{weather}}(k), D_{\text{dwelling}}(k), D_{\text{cascade}}(k-1)\big) \tag{1}$$

$$D_{\text{total}}(k) = D_{\text{traffic}}(k) + D_{\text{weather}}(k) + D_{\text{dwelling}}(k) + D_{\text{cascade}}(k-1) \tag{2}$$

B. Target Variables

The system addresses two complementary learning tasks. The first is **continuous delay regression** (Y_{delay}). For any scheduled trip-arrival event at a stop, the delay in minutes is the difference between the actual arrival timestamp (T_{actual}) and the scheduled arrival timestamp ($T_{\text{scheduled}}$):

$$Y_{\text{delay}} = \frac{T_{\text{actual}} - T_{\text{scheduled}}}{60} \quad \text{(minutes)} \tag{3}$$

A positive value ($Y_{\text{delay}} > 0$) denotes a late arrival, a value near zero denotes an on-schedule arrival, and a negative value denotes an early arrival.

The second task is **multi-class disruption severity classification** (Y_{class}), which maps each event into one of four operational states for dispatch management (Table III).

C. Zero-Target-Leakage Protocol

A critical flaw in many published transit machine learning studies is target leakage—accidentally using features that would not be available in a real-time production setting when predicting future delays. To prevent leakage, strict feature segregation rules were enforced (Table IV).

TABLE III
MULTI-CLASS DISRUPTION SEVERITY TARGET (Y_{CLASS})

Class	Condition
NORMAL (0)	$Y_{\text{delay}} \leq 3.0$ min
DELAYED (1)	$3.0 < Y_{\text{delay}} \leq 10.0$ min
SEVERELY_DELAYED (2)	$10.0 < Y_{\text{delay}} \leq 25.0$ min
ROUTE_CHANGED (3)	$Y_{\text{delay}} > 25.0$ min, or incident flag act

TABLE IV
ZERO-TARGET-LEAKAGE FEATURE RULES

Included (valid predictors)	Excluded (leakage risk)
Scheduled arrival timestamp	Actual arrival timestamp (direct target link)
Route ID, stop ID, stop sequence	Disruption severity label (derived from delay)
Scheduled hour and minute	Future stop records for the same trip (temporal leakage)
Cyclical sin/cos time encodings	Unobserved real-time passenger count at future stops
Day of week, weekend, holiday	Post-trip total duration
Hourly weather (temperature, rain, code)	—
Road congestion index and speed	—
Upstream stop $(k-1)$ delay lag	—

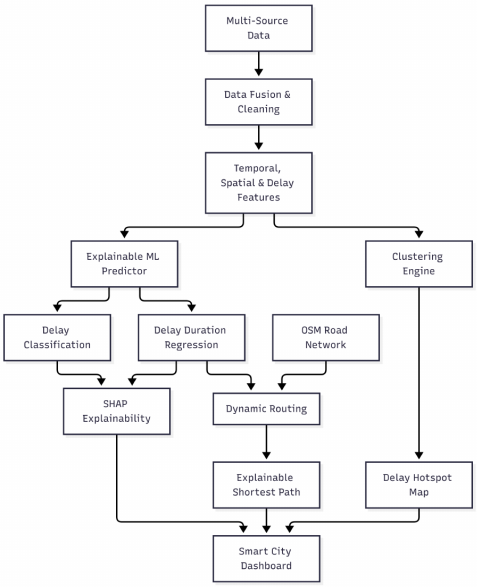


Fig. 1. End-to-end system architecture overview.

D. System Architecture Overview

The proposed architecture consists of five sequential processing stages, from raw municipal data ingestion through to dynamic route optimization (Fig. 1).



Fig. 2. Kimball star schema data warehouse.

E. Data Fusion and Kimball Star Schema

The project integrates 13 disparate municipal and environmental data feeds into a central relational data warehouse following the Kimball dimensional-modelling approach [7]. The warehouse is physically implemented in SQLite (`bmtc_data_warehouse.db`) and comprises the following schema objects (Fig. 2):

- **Central fact table** (`fact_integrated_bus_operations`): 729,648 trip-arrival event records, including scheduled time, actual time, delay minutes, passenger counts, weather, traffic congestion and foreign keys to all dimension tables.
- **Bus fleet dimension** (`dim_bus`): bus ID, type (Non-AC Supra or Volvo AC Celestial), fuel type (Diesel or EV), seating capacity and manufacturer year.
- **Transit stop dimension** (`dim_stop`): name, latitude, longitude, zone, suburban area and shelter facilities of every stop.
- **Calendar dimension** (`dim_calendar`): official Karnataka state holidays, day of week, month, quarter and weekend indicator.
- **Route dimension** (`dim_route`): route code, distance in kilometres, origin and destination terminals and total number of scheduled stops.

F. Data Preprocessing and Imputation

Raw transit telemetry streams frequently suffer from missing sensor packets, GPS signal drops and outlier spikes. To prepare the data for model training:

- **KNN imputation for sensor dropouts.** A K-Nearest Neighbours imputer ($K = 5$) imputes missing environmental and traffic metrics. Experimental testing confirmed that the system maintains high prediction accuracy even under simulated 15% random sensor packet loss.
- **Robust scaling.** `RobustScaler`, which removes the median and scales by the interquartile range (IQR), is applied to continuous parameters such as temperature, traffic speed and delay minutes, so that extreme congestion outliers do not distort model weights.

TABLE V
STRICT TEMPORAL DATA PARTITIONING

Partition	Period	Share	Rows	Purpose
Training	2022–2024	70%	∼510,750	Fit scalers, encoders and learn model weights
Validation	2025	15%	∼109,450	Hyperparameter tuning and early stopping
Out-of-time test	2026 (Jan–Aug)	15%	∼109,448	Holdout benchmark for real-world generalization

- **One-hot categorical encoding.** `OneHotEncoder` converts categorical features (bus type, fuel type, zone and area) into sparse binary vectors.

G. Feature Engineering

To maximize predictive power without target leakage, three families of feature transformations were implemented.

1) Cyclical trigonometric time encodings: Integer representations of time (e.g., hour 0–23) create an artificial discontinuity between 23:59 and 00:00. To preserve continuity, scheduled months m and hours h are converted into two-dimensional sine/cosine features (stored as `month_sin`, `month_cos`, `hour_sin`, `hour_cos`):

$$m_{\sin} = \sin\left(\tfrac{2\pi m}{12}\right), \quad m_{\cos} = \cos\left(\tfrac{2\pi m}{12}\right) \tag{4}$$

$$h_{\sin} = \sin\left(\tfrac{2\pi h}{24}\right), \quad h_{\cos} = \cos\left(\tfrac{2\pi h}{24}\right) \tag{5}$$

2) Upstream trajectory lag delay: To model sequential delay propagation without complex graph dependencies, the actual or predicted delay at the previous stop $(k-1)$ on the same trip instance is injected as an input feature when forecasting delay at stop k: `lag_delay_prev_stop = Delay_at_Stop(k-1)`.

3) Peak-hour and weather-severity indicators: `is_peak_hour` is set to 1 during the morning peak (08:00–11:00) and evening peak (17:00–20:00), and 0 otherwise. `weather_severity_index` is a combined scalar obtained by multiplying precipitation volume by weather-code severity.

H. Strict Temporal Data Splitting

Random cross-validation splits cause temporal leakage by using future data to predict past events. To reflect real-world deployment, the five-year dataset was partitioned strictly by time (Table V).

I. Machine Learning Model Architectures

1) Tabular champion: XGBoost regressor: Gradient Boosted Decision Trees (XGBoost) [2] build an ensemble of trees sequentially, with each new tree correcting the residual errors of earlier ones. The key hyperparameters are: 500 estimators with early stopping (patience $=$ 20), learning rate 0.05, maximum depth 8, subsample 0.8 and `colsample_bytree` 0.8.

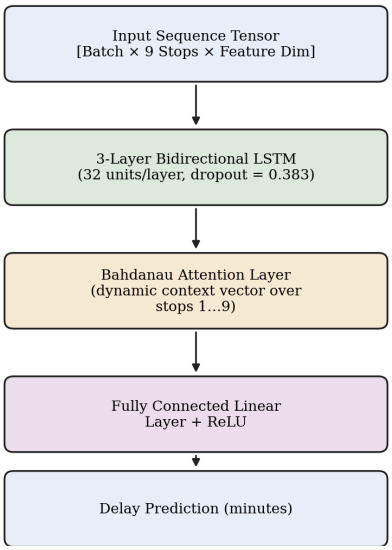


Fig. 3. Bidirectional LSTM with Bahdanau attention architecture.

2) Sequential champion: PyTorch BiLSTM with Bahdanau attention: Urban transit delays are naturally sequential, whereas conventional algorithms treat individual bus stops as separate data points. A Bidirectional LSTM [9] network processes the transit sequence in both forward and backward directions to capture spatial–temporal propagation and to model how downstream traffic influences upstream journey times. Because not every stop contributes equally to a vehicle's overall delay, a Bahdanau attention mechanism [3] is integrated: it computes dynamic alignment weights (α_i) over the sequence, letting the network concentrate on critical bottlenecks before the optimized context vector is passed through a fully connected layer for the final delay prediction (Fig. 3). The network is implemented in PyTorch [12].

- **Architecture:** 3 bidirectional LSTM layers with 32 hidden units per layer (31,457 trainable parameters).
- **Dropout:** 0.383 applied between layers to prevent overfitting.
- **Attention:** alignment weights (α_i) over all 9 stop-sequence positions in a trip, focusing on the bottleneck stops that contribute most to downstream delays.
- **Optimizer:** AdamW (learning rate 1.54×10^{-3}, weight decay 1×10^{-4}).

3) Classification models: For multi-class disruption severity classification (NORMAL, DELAYED, SEVERELY_DELAYED, ROUTE_CHANGED), a cost-sensitive multi-class XGBoost model (`multi:softprob`) and a balanced Random Forest classifier (scikit-learn [11]) were trained with class-weight balancing to handle rare disruption incidents.

J. Software Architecture and Modular Pipeline

The software is built as a modular, eight-step execution pipeline in three logical phases, to ensure scalability and repro-

ducibility. In **Data Preparation (Steps 01–02)**, over 729,000 telemetry rows are audited to prevent leakage, and 3-D tensors and spatial–temporal features are engineered for deep learning. **Predictive Modelling (Steps 03–06)** implements the BiLSTM with attention and the XGBoost regressor for continuous delay forecasting, while secondary classifiers assign severity levels; SHAP and Gini importance metrics are extracted throughout this phase. **Operational Application (Steps 07–08)** benchmarks A* and Dijkstra routing by inserting predicted delays into a live OpenStreetMap graph. A master script unifies all measurements into single reports and visualizations, and the pipeline runs end to end in under 180 seconds.

K. Interactive Data Warehouse Explorer

An interactive terminal tool (`show_data_warehouse.py`) lets reviewers and database administrators examine the star-schema warehouse. It runs live SQL queries against `bmtc_data_warehouse.db` and prints table schemas, dimension joins, row counts and analytical view outputs (`vw_route_performance`) to the console.

L. Dynamic Route Optimization Engine

When traffic is disrupted, a transit system must find alternative routes that reduce trip time without sacrificing computational efficiency. The approach uses OpenStreetMap (OSM) data downloaded with the OSMnx library [6] to represent the road network as a directed graph of intersections and connected road segments, each edge carrying a travel-time cost. Only primary, secondary, tertiary, highway and trunk roads suitable for bus service were included. The engine was evaluated with both Dijkstra's algorithm [5] and A* search under three simulated disruption scenarios.

1) Dynamic edge cost formulation: Trip time is the edge weight for each road section; the initial travel time is derived from road length and expected edge speed. The network is the graph

$$G = (V, E), \tag{6}$$

where V is the set of road-network nodes and E the connecting road segments. The cost assigned to edge e is

$$W(e) = T_{\text{travel}}(e), \tag{7}$$

where $T_{\text{travel}}(e)$ is the predicted trip time. To simulate disruptions, the travel-time costs of affected segments are dynamically increased: in *Micro Routing*, a severe accident is simulated by multiplying the travel time of the impacted segments by 100; in *Macro Detour*, edges in a chosen area receive a 50-fold cost; and in *Extreme Gridlock*, roughly half of all road edges are randomly assigned a five-fold cost.

2) Dijkstra's algorithm: Dijkstra's algorithm finds the least-cost path from the source to the destination by maintaining the shortest known cost to each node, using a priority queue to select the node with the lowest cumulative trip time until the destination is reached. Here it uses the dynamically modified `travel_time` attribute as the edge weight, so a segment lengthened by traffic or an incident is bypassed whenever an alternative has lower total cost.

3) A search:* A* search was implemented as a second routing algorithm for comparison. Unlike Dijkstra, A* adds a heuristic estimate of the remaining cost to the destination:

$$f(n) = g(n) + h(n), \tag{8}$$

where $f(n)$ is the overall estimated cost, $g(n)$ the accumulated cost from the source to node n, and $h(n)$ the anticipated cost from n to the destination. The heuristic converts the Haversine distance between the current node and the destination into an estimated travel time.

4) Experimental scenarios:

- **Scenario 1 – Macro Detour:** certain road portions receive much higher travel-time costs, causing a significant disruption between Hebbal and Silk Board. The goal is to see whether the algorithms find a practical diversion around a heavily crowded area.
- **Scenario 2 – Micro Routing:** a localized disruption between Nagavara and Hennur, such as an accident or road closure, is represented by a factor of 100 on the originally chosen route, forcing a different local path.
- **Scenario 3 – Extreme Gridlock:** widespread congestion is modelled by giving about half of the road edges five times their original travel duration using a deterministic random seed, testing performance when congestion affects a large part of the network.

M. Explainable AI and Natural-Language Detour Cards

The routing pipeline incorporates SHAP [4] to turn black-box outputs into transparent recommendations. For every delay forecast, SHAP computes the exact feature attributions ϕ_i that sum to the predicted delay:

$$\hat{Y}_{\text{delay}} = \text{BaseValue} + \sum_{i=1}^{M} \phi_i \tag{9}$$

These attributions are parsed to generate human-readable Detour Explanation Cards in the commuter interface, for example: "Bypassing segment Marathahalli Bridge → Bellandur EcoSpace. Reason: High delay risk (+18.4 mins) driven primarily by Traffic Congestion Index (+10.2 mins) and Heavy Rainfall (+6.1 mins)." Fig. 4 illustrates the waterfall of contributions behind such a card.

N. Dual-Perspective Interactive Dashboard

The front end is built with Streamlit, Plotly and Folium and provides two portals:

- **Commuter portal:** passengers monitor expected arrival times and origin/destination stops, view delay severity badges (LOW, MEDIUM, HIGH) and navigate interactive dynamic detour maps with SHAP explanation cards.
- **Transit planner portal:** designed for municipal bus operators, it shows an interactive heatmap of physical congestion hotspots using DBSCAN spatial clustering [8] on the GPS coordinates of repeated delays. It also generates automated schedule-buffer recommendations, e.g., "Route 500D experiences recurrent 12-minute delays at

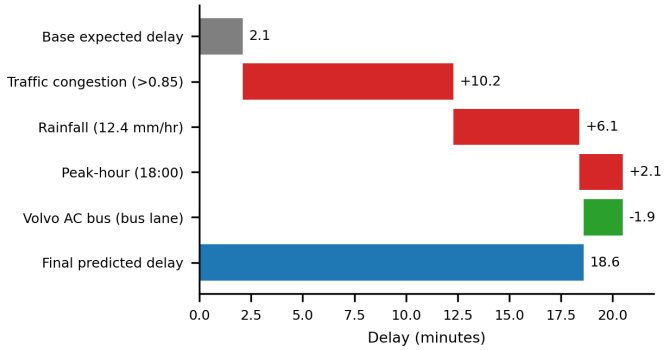


Fig. 4. SHAP explainability waterfall: base delay 2.1 min plus feature contributions give a final predicted delay of 18.6 min (card: "Route delayed by 18.6m due to Traffic and Rain").

TABLE VI
ESTIMATED MONTHLY CLOUD COST AT PRODUCTION SCALE

Component	USD/month
AWS RDS PostgreSQL/PostGIS (500 GB)	∼350
3-node Apache Kafka cluster (1,200 queries/s)	240
GCP Cloud Run: FastAPI + XGBoost inference	180
AWS 8 GB Redis cache (real-time tracking)	110
Outgoing bandwidth (∼1 TB WebSocket data)	∼90
Total	**∼970**

Silk Board during evening peak hours. Recommendation: add 5 minutes buffer time or deploy 1 backup bus."

O. Cloud Engineering and Financial Feasibility

A technical analysis compared running a local demo with deploying the system on production cloud infrastructure. For research and prototyping the system runs locally on a laptop, but a production deployment on local hardware for thousands of daily users introduces critical bottlenecks:

- **Memory constraints:** road-graph storage (OSRM) and spatial caching (Redis) use gigabytes of RAM during live 5-second GPS pings from 6,000 buses, quickly causing out-of-memory crashes.
- **Network egress limits:** serving real-time WebSocket coordinates to thousands of mobile devices requires enterprise bandwidth rather than home internet connections.
- **Database locking:** file-level locking on write operations causes "database locked" exceptions under concurrent writes from thousands of IoT-equipped buses.

A citywide platform tracking 6,000 active buses and over 50,000 passengers needs a robust, enterprise-grade cloud architecture. The core software stack (PostgreSQL, Kafka, FastAPI, Redis) is open source and free of licensing fees, but the hardware infrastructure incurs significant operating cost: the predicted analysis requires a monthly cloud budget of roughly USD 970 (Table VI). The spatial database layer is the largest expense, since 500 GB of geographic data requires a powerful AWS RDS machine for PostgreSQL/PostGIS.

TABLE VII
CONTINUOUS DELAY REGRESSION BENCHMARK RESULTS

Metric	XGBoost Regressor	BiLSTM + Attention
Category	Tabular champion	Sequential champion
RMSE (min)	1.631	**0.865**
MAE (min)	1.298	**0.346**
R^2 score	0.9859	**0.9970**
Accuracy (±3 min)	93.2%	**98.1%**
Train time (s)	**42.3**	427.5

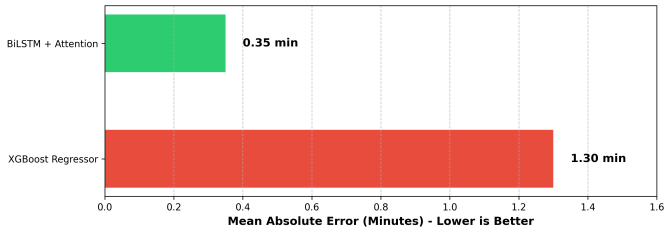


Fig. 5. Regression performance comparison (MAE; lower is better).

TABLE VIII
DISRUPTION CLASSIFICATION BENCHMARK RESULTS

Metric	XGBoost Classifier	Random Forest
Role	Primary model	Baseline comparison
Macro F1	**0.7511**	0.7489
Weighted F1	**0.8873**	0.8856
Balanced accuracy	**0.8849**	0.8773
Accuracy	**87.9%**	87.7%
Train time (s)	185.3	**105.7**

IV. RESULTS

A. Data Audit and Zero-Leakage Validation

The dataset was automatically profiled with `data_audit_leakage_checks` before model training. The audit covered 729,648 rows across 35 columns. Missing values were handled using KNN imputation and were strictly confined to `holiday_name` (expected for non-holiday days). Automated assertions confirmed the 100% exclusion of `actual_arrival`, `disruption_label` and target-derived columns from the feature matrices (X).

B. Delay Regression Benchmarks

Models were assessed on the 2026 out-of-time test set (109,448 unseen operational instances) using four measures: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), the coefficient of determination (R^2) and accuracy within a ±3-minute window (Table VII, Fig. 5).

C. Disruption Severity Classification Benchmarks

The multi-class classifiers predicting the operational severity classes (NORMAL, DELAYED, SEVERELY_DELAYED, ROUTE_CHANGED) were evaluated as shown in Table VIII.

D. Routing Algorithm Comparison

Table IX summarizes the routing engine's experimental outcomes for Dijkstra and A* under the three disruption scenarios, and Fig. 6 visualizes the resulting routes. Both algorithms found the same optimal trip-time cost in every scenario; they differ in latency and search-space size.

A* consistently examined fewer nodes because its heuristic focuses the search on the destination: 1,688 versus 3,324 nodes (Macro Detour), 43 versus 169 (Micro Routing) and 3,471 versus 3,938 (Extreme Gridlock). In Fig. 6, the A* search footprint is much smaller and more directed than the large, circular expansion of Dijkstra. Despite visiting more nodes, Dijkstra recorded lower execution latency in every scenario: 23.09 ms versus 23.57 ms (Macro Detour), 3.86 ms versus 4.70 ms (Micro Routing) and 31.22 ms versus 52.46 ms (Extreme Gridlock). Micro Routing produced the largest saving (553.01 min) and Extreme Gridlock a saving of 30.50 min; in every case both algorithms returned equal optimized route costs.

E. Dynamic Route Optimization Case Study

To assess practical impact, a simulation was run along one of Bengaluru's busiest arterial routes, from Hebbal bus stop (BS_HEB) to Central Silk Board Junction (BS_SIL) via the Outer Ring Road (Table X, Fig. 7).

F. Execution Speed Profiling

Profiling confirmed ultra-low latency suitable for real-time deployment: Dijkstra pathfinding takes 0.0658 ms per route query, XGBoost inference 0.0027 ms per sample and BiLSTM inference 0.0614 ms per sequence.

V. Discussion

A. Regression Performance

The BiLSTM with attention was clearly superior: for 98.1% of test trips it predicted arrival time within ± 3 minutes, with MAE $= 0.35$ min and $R^2 = 0.9970$. This performance arises because the attention mechanism captures route-sequence memory that tabular models overlook by tracking cumulative delay accumulation across successive stops. The tabular XGBoost regressor nevertheless remains attractive: it trained in just 42.3 s (about ten times faster than the BiLSTM) while still reaching MAE $= 1.30$ min and $R^2 = 0.9859$, making it a lightweight backup model for resource-constrained edge devices.

B. Classification Performance

Both classifiers achieved good balanced accuracy (about 88.5%) and high overall accuracy (about 87.9%). With a higher macro F1 of 0.7511, the cost-sensitive XGBoost classifier performed better than Random Forest on the infrequent disruption classes (ROUTE_CHANGED and SEVERELY_DELAYED), which supports class-weighted boosting for imbalanced disruption data.

C. Routing Impact

The static scheduled route took 108.97 minutes because of heavy rain traffic on the Outer Ring Road. By ingesting machine-learning delay projections, the dynamic Dijkstra engine found an open alternate route via Majestic and BTM Layout and reduced travel time to 26.67 minutes—a saving of 82.3 minutes (75.53%). This shows that delay-weighted graph routing can substantially improve commuter efficiency amid significant disruptions. The algorithm comparison shows a trade-off: A* narrows the search space, whereas Dijkstra avoids the overhead of repeatedly computing the Haversine heuristic and so records lower latency, with identical route quality.

D. Explainability and Operational Value

SHAP-based explanations convert transparent delay projections into human-readable detour recommendations for both commuters and operators, addressing the "black-box" gap identified in Section II-E. The dual-perspective dashboard additionally gives operators DBSCAN-derived congestion hotspots and schedule-buffer advice.

E. Deployment Considerations and Limitations

The cloud analysis shows that a citywide deployment is feasible at roughly USD 970 per month, but that local hosting is limited by memory, network egress and database locking. As stated in Section I-C, the present implementation is a research prototype: it covers selected Bengaluru corridors, does not include live streaming ingestion, and does not automate fleet dispatch. The route-optimization results come from simulated disruptions on OSM graphs rather than live operations. Future work includes Graph Neural Network-based spatial modelling of the whole route network, live data-stream ingestion and reinforcement-learning-based fleet dispatch optimization.

VI. Conclusion

This project built and implemented an end-to-end, enterprise-grade intelligent transportation system for predicting public bus delays and dynamically optimizing commuter routes in Bengaluru. It addresses the fragmentation and data-quality gaps found in earlier literature by combining 13 heterogeneous municipal data streams—GTFS schedules, AVL GPS telemetry, weather, traffic congestion indices and passenger occupancy—into a Kimball star-schema data warehouse of 729,648 arrival records.

The dual prediction engine shows that a sequential deep model (BiLSTM with Bahdanau attention) significantly outperforms a tabular gradient-boosted model on continuous delay estimation (MAE 0.35 versus 1.30 minutes), while the lightweight XGBoost regressor remains a strong low-latency backup for constrained deployments. Cost-sensitive classifiers distinguish disruption severity with 87.9% accuracy, enabling proactive dispatch decisions.

Feeding these predictions into a dynamic Dijkstra-based routing engine over OpenStreetMap graphs delivered a quantifiable benefit: an 82.3-minute (75.5%) reduction in travel

TABLE IX
DIJKSTRA VS. A* SEARCH UNDER DIFFERENT TRAFFIC SCENARIOS

Scenario	Algorithm	Actual time (min)	Optimized time (min)	Time saved (min)	Latency (ms)	Nodes searched
Macro Detour	A*	25.54	25.54	0.00	23.57	1,688
Macro Detour	Dijkstra	25.54	25.54	0.00	23.09	3,324
Micro Routing	A*	626.81	73.80	553.01	4.70	43
Micro Routing	Dijkstra	626.81	73.80	553.01	3.86	169
Extreme Gridlock	A*	92.94	62.43	30.50	52.46	3,471
Extreme Gridlock	Dijkstra	92.94	62.43	30.50	31.22	3,938

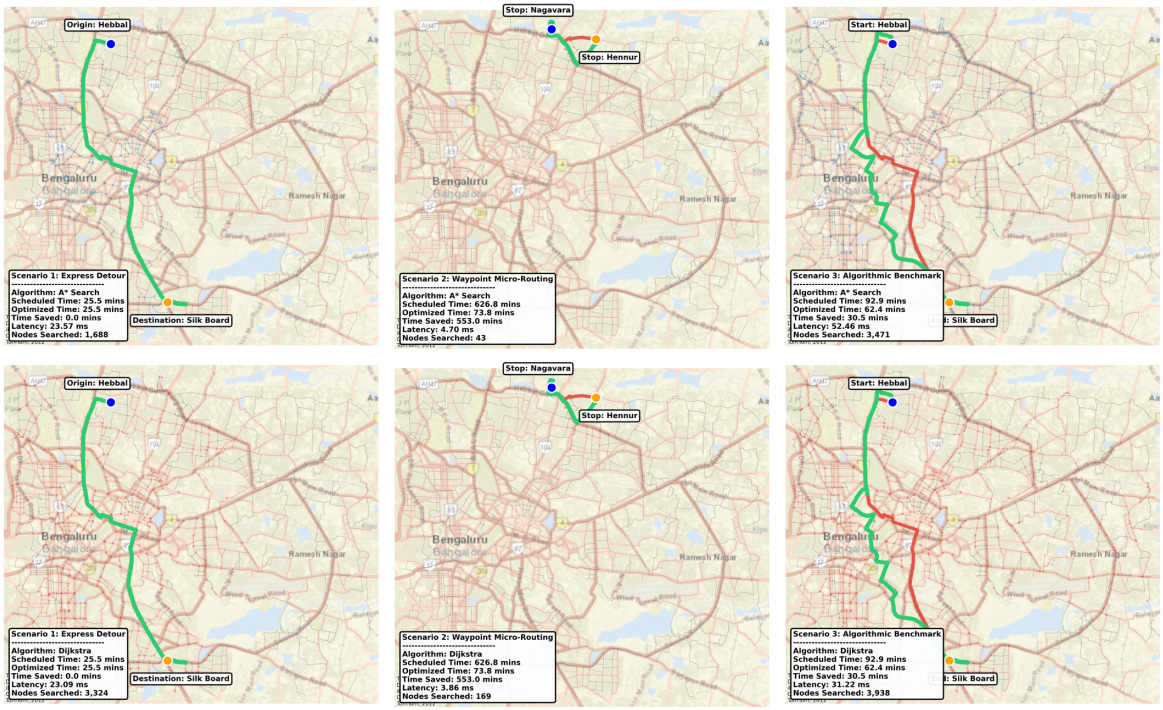


Fig. 6. Route visualizations for A* search (top row) and Dijkstra's algorithm (bottom row) under (left to right) Macro Detour, Micro Routing and Extreme Gridlock scenarios. Searched nodes: A* 1,688 / 43 / 3,471; Dijkstra 3,324 / 169 / 3,938.

TABLE X
ROUTE OPTIMIZATION CORRIDOR EXPERIMENTAL RESULTS

Metric	Scheduled congested path	ML dynamic detour path
Route taken	Hebbal → Nagavara → Hennur → KR Puram → Marathahalli → Bellandur → Agara → Silk Board	Hebbal → Majestic KBS → Jayanagar 4th Block → BTM Layout → Silk Board
Total travel time	108.97 min (1.82 h)	26.67 min (0.44 h)
Time saved	—	82.30 min (75.53% reduction)

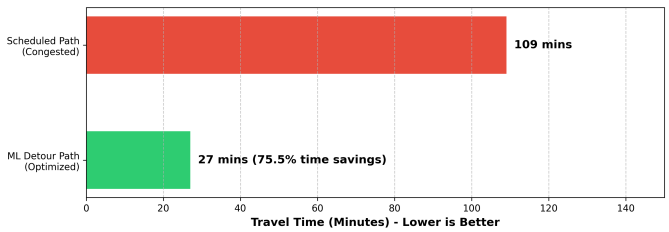


Fig. 7. Route optimization travel-time comparison.

time on a highly congested path under simulated severe disruption. SHAP-based explainable AI filled a crucial research gap by converting delay projections into human-readable diversion explanations for commuters and operators.

All three project objectives—precise multimodal delay prediction, explainable dynamic route optimization and a dual-perspective decision-support system—were achieved, and the cloud cost analysis provides a financially grounded roadmap for scaling the prototype into a production municipal transit platform. Future work could add live streaming ingestion, Graph Neural Network modelling of the full route network, and reinforcement-learning-driven fleet dispatch.

ACKNOWLEDGMENT

The authors thank CHRIST (Deemed to be University) and the School of Engineering and Technology for their patronage: Vice Chancellor Dr. Rev. Fr. Jose C. C.; Pro Vice Chancellor Dr. Rev. Fr. Viju P. D.; Director Dr. Rev. Fr. Jiby Jose E.; Assistant Director Rev. Fr. Shijin P. J.; Dean Dr. Raghunandan Kumar R; and Associate Dean Dr. E. A. Mary Anita. They are grateful to Dr. M. Balamurugan, Head of the Department of Computer Science and Engineering, for the opportunity to undertake this project, and to their guide, Dr. Karthikeyan H, whose constant monitoring and encouragement helped keep the project on schedule. They also thank the teaching and non-teaching staff of the department, and their families and friends for their support.

REFERENCES

[1] General Transit Feed Specification (GTFS), "GTFS Schedule Reference and Static Data Guidelines," MobilityData International, 2022. [Online]. Available: https://gtfs.org/

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2016, pp. 785–794.

[3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

[4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

[5] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, no. 1, pp. 269–271, 1959.

[6] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Comput., Environ. Urban Syst.*, vol. 65, pp. 126–139, 2017.

[7] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discovery Data Mining (KDD)*, 1996, pp. 226–231.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] Bengaluru Metropolitan Transport Corporation (BMTC), "Annual operational performance & Intelligent Transport System (ITS) report," Government of Karnataka, India, 2023.

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[12] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

TABLE XI
GLOSSARY OF ABBREVIATIONS

Term	Expansion / definition
AVL	Automatic Vehicle Location
GTFS	General Transit Feed Specification
ITS	Intelligent Transportation System
XAI	Explainable Artificial Intelligence
SHAP	Shapley Additive Explanations
LSTM	Long Short-Term Memory (neural network)
BiLSTM	Bidirectional Long Short-Term Memory
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
R^2	Coefficient of determination
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
OSM / OSMnx	OpenStreetMap / Python toolkit for OSM street networks
APC	Automated Passenger Counter
KNN	K-Nearest Neighbours (imputation algorithm)
OOT	Out-of-Time (test partition)